

一、CUDA 编程基础

Why CUDA

在 PyTorch 层面，执行一次矩阵乘法仅仅需要调用 $C = A @ B$ 。但在 AI Infra 中，背后是数百 GB/s 的显存数据搬运和数以万计的并发线程调度。

大模型的训练和推理优化（如 FlashAttention 减少显存读写、W8A8 量化调用 Tensor Core 加速），其本质都是**压榨 GPU 的硬件特性**。因此，理解 CUDA 与 GPU 软硬映射，是跨入 AI Infra 领域的第一道门槛。

从 CPU 到 GPU：执行模型与软硬映射

要理解 CUDA，首先要理解 GPU 的设计哲学：**吞吐量至上 (Throughput-oriented)**。CPU 拥有少量强大的核心，擅长复杂逻辑分支，而 GPU 拥有海量简单的核心，擅长高度并行的重复计算。

在 CUDA 编程中，我们将 CPU 称为 **主机 (Host)**，GPU 称为 **设备 (Device)**。CUDA 抽象出了一套极具美感的线程层级结构，并与 GPU 硬件映射：

1. 线程 (Thread) 与 SP (CUDA Core)

- **软件层**：最基础的执行单元是 Thread。
- **硬件层**：每个 Thread 实际上运行在 GPU 的一个 SP（流处理器 / CUDA Core）上。

2. 线程块 (Block) 与 SM (流多处理器)

- **软件层**：多个 Thread 组成一个 Block。同一个 Block 内的线程可以进行数据同步，并共享一块极速的内存。
- **硬件层**：一个 Block 会被整体调度到 GPU 的一个 **SM (Streaming Multiprocessor)** 上执行。SM 是 GPU 最核心的计算单元，包含了多个 CUDA Core、寄存器和共享内存。

3. 网格 (Grid) 与 GPU 设备

- **软件层**：多个 Block 组成一个 Grid。一个 Grid 对应一次完整的函数调用。
- **硬件层**：Grid 被分配给整个 GPU 芯片执行。

(注：在微观执行上，GPU 会将 32 个线程捆绑成一个 **Warp (线程束)**，这是 GPU 调度的最小基本单位，所有 32 个线程必须严格执行相同的指令，即 SIMT 架构。)

CUDA 初探：如何编写一个 Kernel

在 CUDA 中，运行在 GPU 上的函数被称为 **核函数 (Kernel)**。

通过一个极简的**向量加法 ($C = A + B$)**，展示底层是如何驱动上万个线程并发计算的。

1. 核心修饰符

- `__global__`：定义一个 Kernel 函数。表示该函数在 CPU (Host) 上调用，但在 GPU (Device) 上执行。
- `__device__`：表示该函数只能在 GPU 上调用和执行。

2. Kernel 函数实现 (CUDA C++)

假设我们有两个长度为 N 的数组相加。在 CPU 上，我们需要写一个 `for` 循环；在 GPU 上，我们消除了循环，让每一个线程只负责计算其中的一个元素。

```
// CUDA Kernel: 向量加法
__global__ void vectorAddKernel(const float* A, const float* B, float* C, int N) {
    // 核心：计算当前线程在整个 Grid 中的全局唯一索引 ID
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // 如果索引没有越界，则执行单步加法
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

- 内置变量解析：

- `blockDim.x`：一个 Block 里有多少个线程（比如 256）。
- `blockIdx.x`：当前线程属于第几个 Block。
- `threadIdx.x`：当前线程在自己 Block 内的编号。
- (这就好比计算小明在学校的绝对座号：所在班级编号 $\times$ 每班人数 + 班内学号)

3. 在 Host (CPU) 上的调度代码

编写完 Kernel 后，我们需要在 CPU 端分配显存、搬运数据并触发执行：

```

void runVectorAdd() {
    // 1. 在 GPU (Device) 上分配显存 (类比 PagedAttention 关注的显存分配)
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    // 2. 将数据从 CPU 内存拷贝到 GPU 显存 (PCIe 带宽瓶颈就在这里)
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // 3. 定义线程层级拓扑: 每个 Block 256 个线程, 计算需要多少个 Block
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    // 4. 启动 Kernel (这就是 CUDA 特有的 <<<grid, block>>> 执行语法)
    vectorAddKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

    // 5. 将计算结果从 GPU 拷贝回 CPU
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
}

```

设备层的核心优化机制

仅仅写出能跑的 Kernel 是不够的, AI Infra 的核心在于“优化”。在编写底层算子 (如 Attention 机制) 时, 必须对 **GPU 的多级内存模型 (Memory Hierarchy)** 深入研究。

1. 全局内存 (Global Memory / HBM) 与 合并访存 (Coalescing)

- **原理**: 全局内存就是我们常说的“显存”(如 A100 的 80G HBM)。它的容量最大, 但读取延迟极高 (约数百个时钟周期)。
- **优化法则: 合并访存 (Memory Coalescing)**。当同一个 Warp 内的 32 个线程去访问一段连续的显存地址时, GPU 硬件会将这 32 次独立的访存操作“打包”成一次大的内存事务。如果你写的代码是跳跃式访问 (比如按列读取行主序矩阵), 带宽利用率会暴跌至 1/32。
- (映射: 我们在后续讲到 *Tensor Parallelism* 切分矩阵时, 必须考虑切分方向以保证底层依旧能进行合并访存。)

2. 共享内存 (Shared Memory) 与 分块计算 (Tiling)

- **原理**: Shared Memory 是位于 SM 内部的一块极速缓存 (SRAM, 通常只有一百多 KB), 延迟极低, 同一个 Block 内的线程可以共享访问它。

- **优化法则：**在进行大规模矩阵乘法（GEMM）或 Attention 计算时，如果我们直接从全局内存读取数据，访存会成为严重瓶颈（Memory-bound）。经典的优化策略是 **Tiling（分块）**：
 - 先让 Block 内的线程协作，将一小块数据从慢速的全局内存搬运到快速的共享内存中。
 - 线程再从共享内存中反复读取数据进行密集计算。
 - 计算完成后，再将一小块结果写回全局内存。
- (映射：大名鼎鼎的 **FlashAttention** 技术，其核心创新点正是利用了 Tiling 思想，通过在 Shared Memory 中完成 Attention 的局部计算，彻底避免了 $O(N^2)$ 级别中间激活值写回全局内存的开销，从而实现数倍提速。)

3. 避免分支发散（Warp Divergence）

- **原理：**正如前文提到，一个 Warp 里的 32 个线程共用一个指令发射器。如果在 Kernel 中写了 if-else 分支，且这 32 个线程中有的进入 if，有的进入 else，GPU 将无法并行执行，而是**必须串行地**先执行完 if 分支，再执行 else 分支，这会导致算力直接折半。
- **优化法则：**在底层算子开发中，必须尽量让同一个 Warp 内的线程执行完全一致的代码路径。

从微观到宏观

理解了 CUDA 中的 Thread 和 SM，就能明白为什么小 Batch Size 推理时 GPU 会陷入“访存瓶颈（Memory-bound）”——少量的计算根本无法产生足够的 Thread 来掩盖数据从 HBM 搬运到 SM 的极长延迟；理解了 Shared Memory 和 Tensor Core，我们就能看懂 FlashAttention 和 W8A8 量化的底层原理。

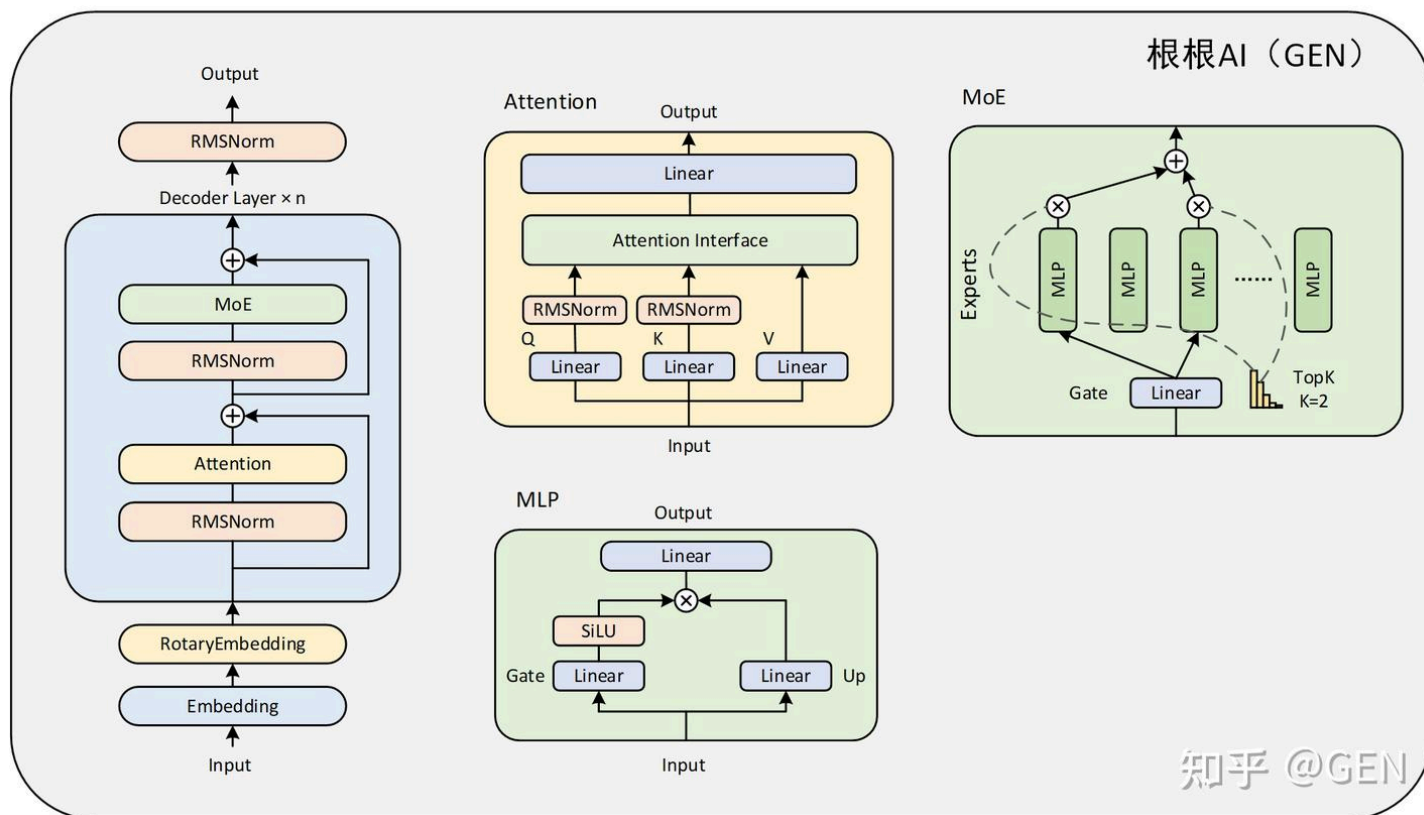
建立起“内存层级”和“并发调度”的微观直觉后，现在可以将视角拉高，正式进入大模型动辄成百上千张卡的宏观世界，去探究的**分布式预训练架构与万级并发推理调度机制**。

二、模型推理加速

推理原理

自回归生成，当前 token 依赖之前已生成的所有 token
以 Qwen3 为例

Qwen3 (MoE)



- 在注意力层内部，通过旋转矩阵修改Query/Key向量

性能分析

1. Prefill (预填充阶段): Compute-bound (算力瓶颈)

- 行为:** 用户输入一段 Prompt 时，模型需要一次性读取并理解这段文本，计算出所有输入 token 的 KV Cache 并输出第一个 token (Time To First Token, TTFT)
- 特性:** 由于输入是一个已知长度的完整序列，模型可以充分利用 GPU 强大的矩阵乘法 (GEMM) 单元进行高度并行的计算。此时数据传输速度跟得上计算速度，系统处于**计算密集型 (Compute-bound)** 状态，GPU 利用率极高

2. Decode (解码阶段): Memory-bound (访存瓶颈)

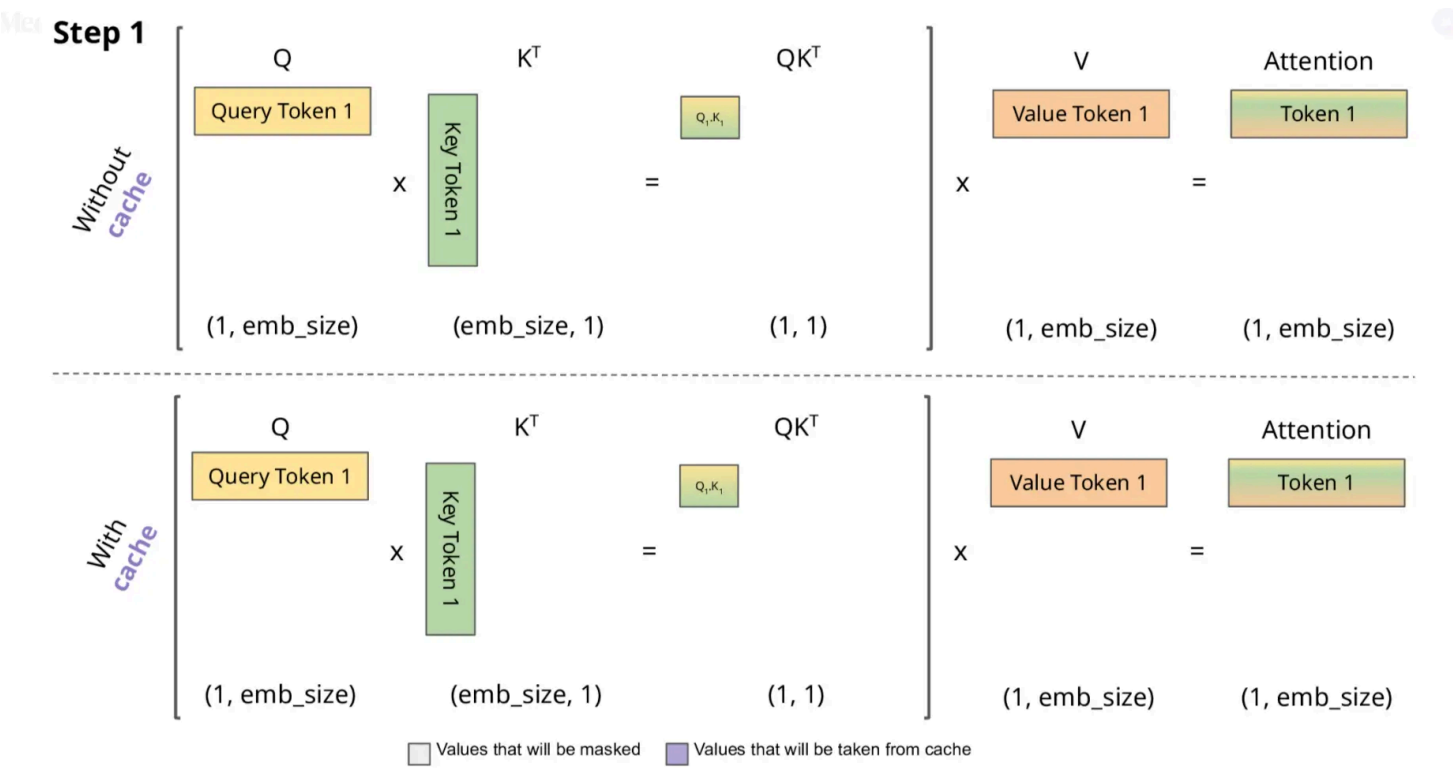
- 行为:** 在输出第一个 token 后，模型进入自回归逐字生成阶段。每生成一个新词，都要读取完整的模型权重以及累积的 KV Cache 进行一次前向传播 (Time Per Output Token, TPOT)
- 特性:** 由于每次输入仅仅是**1个 token**，矩阵-矩阵乘法退化为了矩阵-向量乘法 (GEMV)。GPU 的计算单元 (ALU) 大量闲置，绝大部分时间都在等待数据从显存 (HBM) 搬运到计算单元 (SRAM)。系统陷入**访存密集型 (Memory-bound)** 状态，硬件算力利用率通常不足 5%

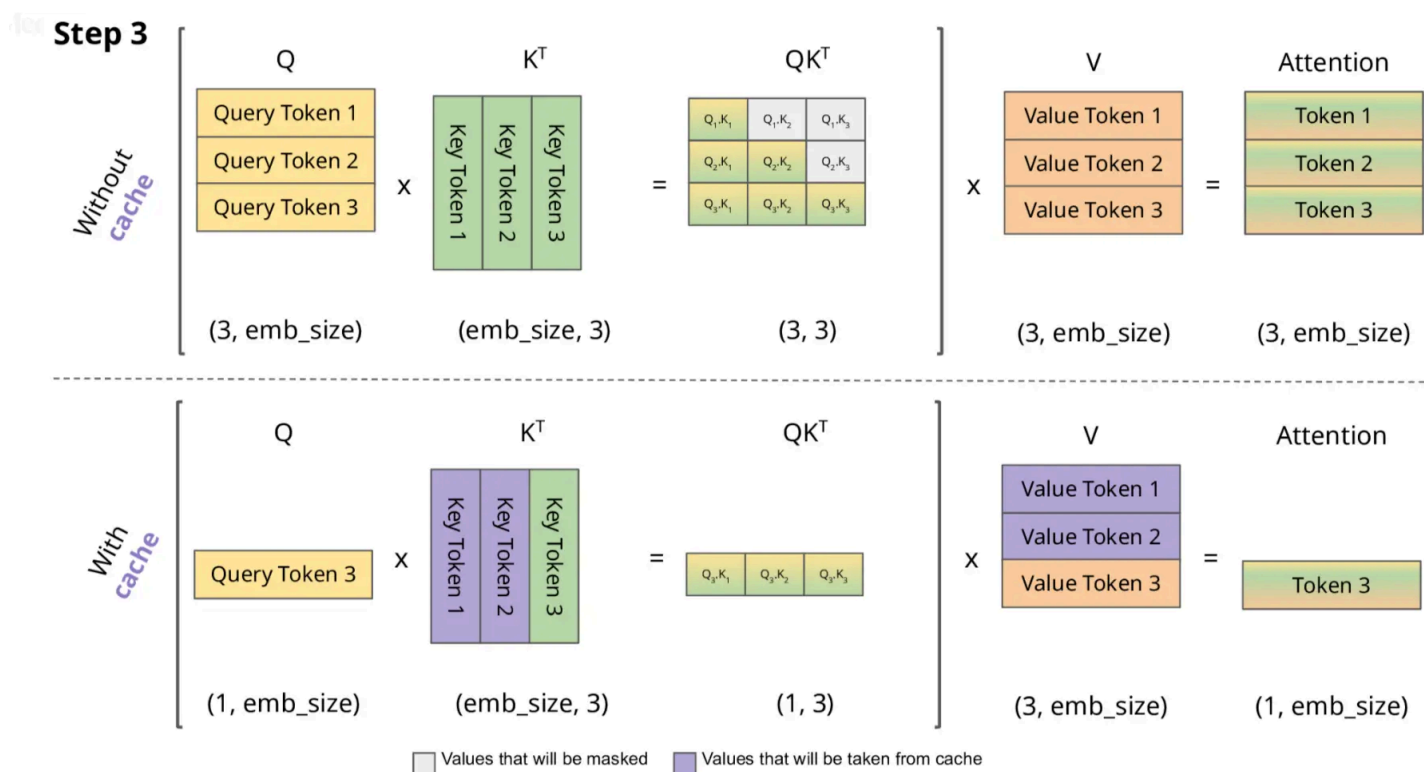
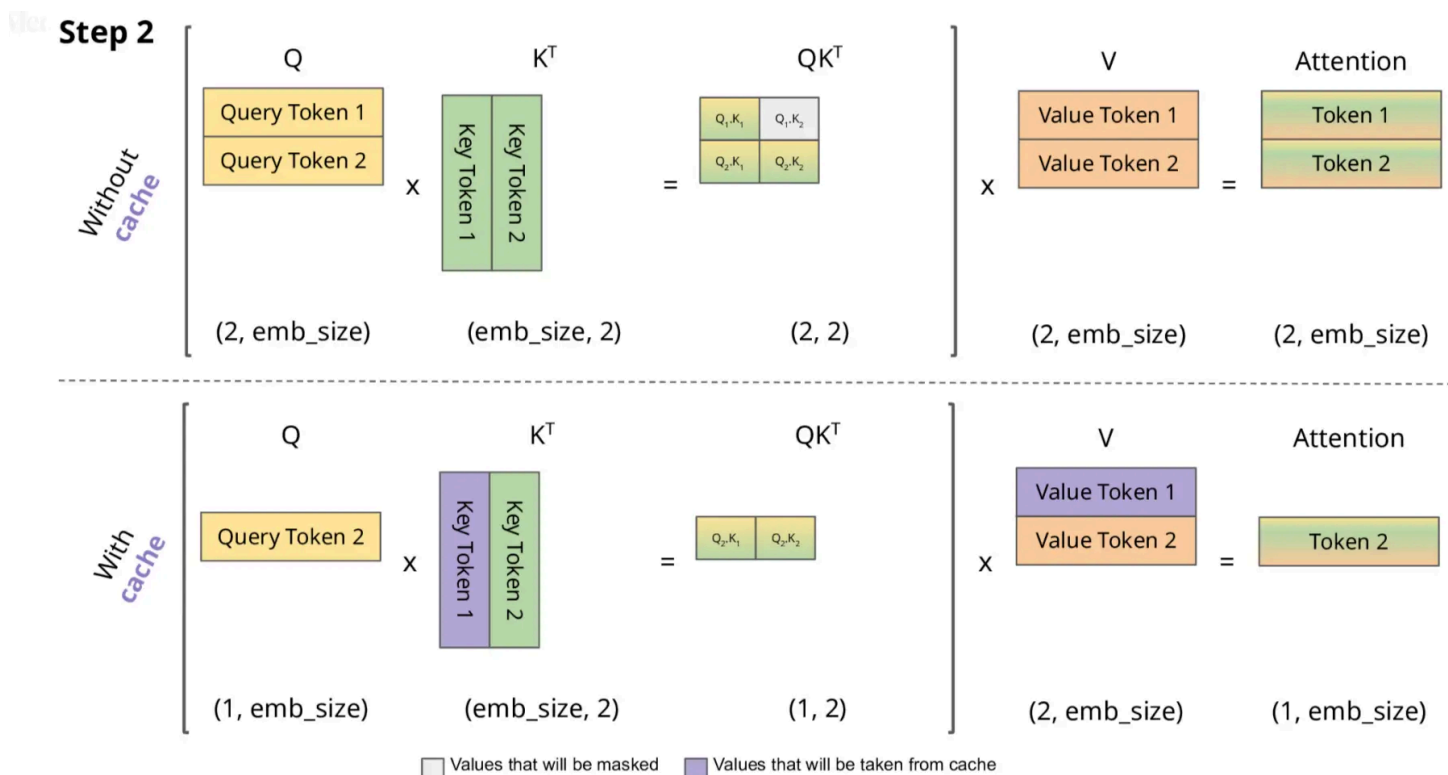
Decode 阶段核心技术

KV Cache

核心思想：缓存历史计算中的键（Key）和值（Value）矩阵，避免重复计算。具体来说：

- 在生成第一个 token 时，模型计算并缓存所有输入 token 的 K 和 V 矩阵
- 生成后续 token 时，只需要计算新 token 的查询（Query）矩阵
- 将新的 Q 矩阵与缓存的 K、V 矩阵进行注意力计算，同时将新 token 的 K、V 追加到缓存中





每次新生成 token 时，只需计算新的 Q 矩阵并与历史 KV 矩阵进行注意力计算
 将时间复杂度从 $O(n^2)$ 降低到 $O(n)$

vLLM - PagedAttention

未使用：进程内部、外部碎片，导致显存利用率低、GPU能同时处理的请求数量严重受限

思想：借鉴操作系统的虚拟内存分页管理机制，将 KV Cache 划分为固定大小的块（Block，例如每个 Block 存放 16 个 token 的 KV 值）

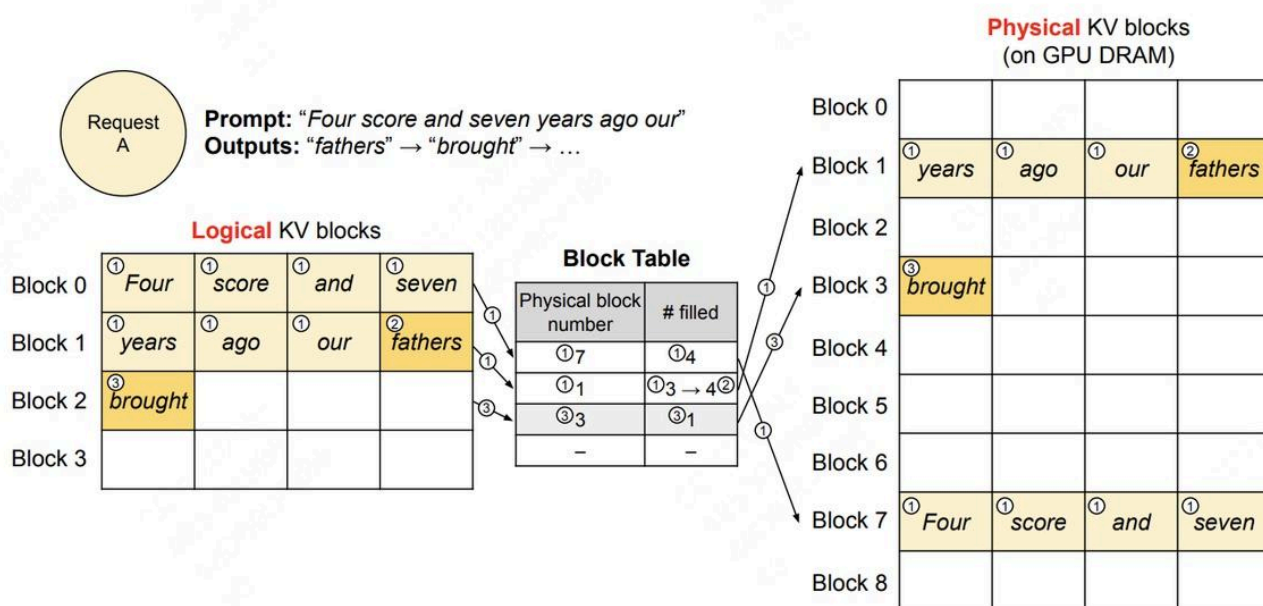


Figure 6. Block table translation in vLLM 知乎 @猛猿

- request：可理解为操作系统中的一个进程
- logical KV blocks（逻辑内存）：操作系统中的虚拟内存，每个block类比于虚拟内存中的一个page。每个block的大小是固定的，在vLLM中默认大小为16，可装16个token的K/V值
- block table：虚拟内存到物理内存的映射表
- physical KV blocks：物理内存，物理块是实际存放KV值的地方，在gpu显存上

(i) Prefill阶段

1. 划分逻辑块：vLLM拿到prompt，先按照设定好的block大小，为prompt划分逻辑块（Logical KV blocks）
2. 划分物理块：将逻辑块映射到物理块中。通过一张block table来记录逻辑块和物理块的映射关系，block table的主要内容包括：逻辑块和物理块的映射关系（physical block number）；每个物理块上被填满的槽位（# filled）
3. 正常计算prompt的KV值，并通过划分好的关系填入物理块中。

(ii) Decode阶段-生成第1个词

1. 使用KV cache计算attention，生成第1个词。逻辑上token都是连续的，物理上数据的存储并不连续

2. 基于新生成的词，更新逻辑块、物理块和block table：对于block table，vLLM将它filled字段由3更新至4
3. 分配新的逻辑块和物理块。当第一个token更新后，逻辑块已装满。所以vLLM将开辟新的逻辑块2，并同时更新对应的block table和物理块

通过动态存储KV cache，来更充分利用gpu显存

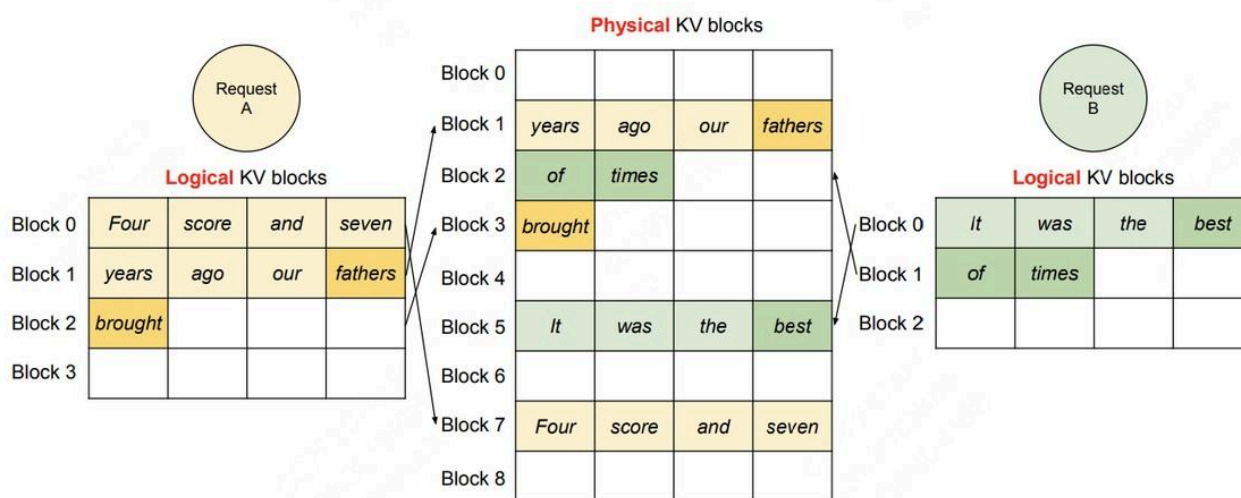


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

知乎 @猛猿

- 不同解码策略：Parallel Sampling、Beam Search
- 调度和抢占

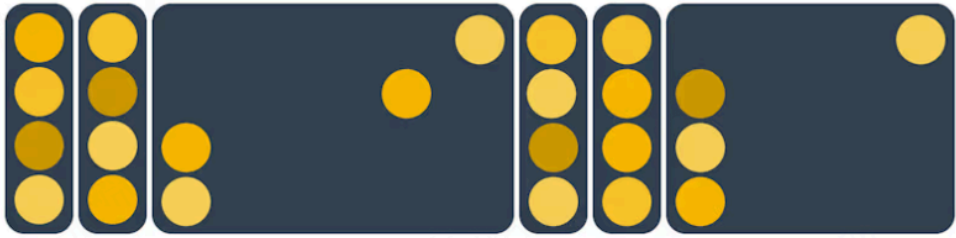
Continuous Batching

Static Batching：请求被分批存放，仅当批次填满时才执行处理

Dynamic Batching：处理该时间段内到达的所有请求。如果批次提前达到数量上限，就会立即启动处理

Dynamic batching for generative model inference

Static
batching



Dynamic
batching



baseten

相关缺点：

Continuous Batching：针对推理中每个 Request 的 Decode 长度不同的特性，当 batch 中的某个序列一旦完成生成，立即被替换为新的请求

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

具体优势：

- Prefill 阶段可以搭载在 Decode 阶段未被充分利用的算力上，提升整体算力利用率。
- Decode 阶段可以和 Prefill 阶段共享一次权重读取，减少内存带宽压力，提高带宽利用率

其余优化

量化

减小模型参数的体积就能直接减少显存搬运的时间

- W8A8 (Weight & Activation 8-bit)：将权重和激活值同时从 16-bit 压缩至 8-bit（例如使用 SmoothQuant 技术平滑激活值的异常点）。且能利用 GPU 专用的 INT8 Tensor Core 进行计算加速。

LLM.int8() 发现当 LLMs 的模型参数量超过 6.7B 的时候，激活中会成片的出现大幅的离群点
引入对角缩放矩阵，把 X 量化的难度转移一部分给 W：

$$Y = (X \cdot S_{smooth}^{-1}) \times (S_{smooth} \cdot W)$$

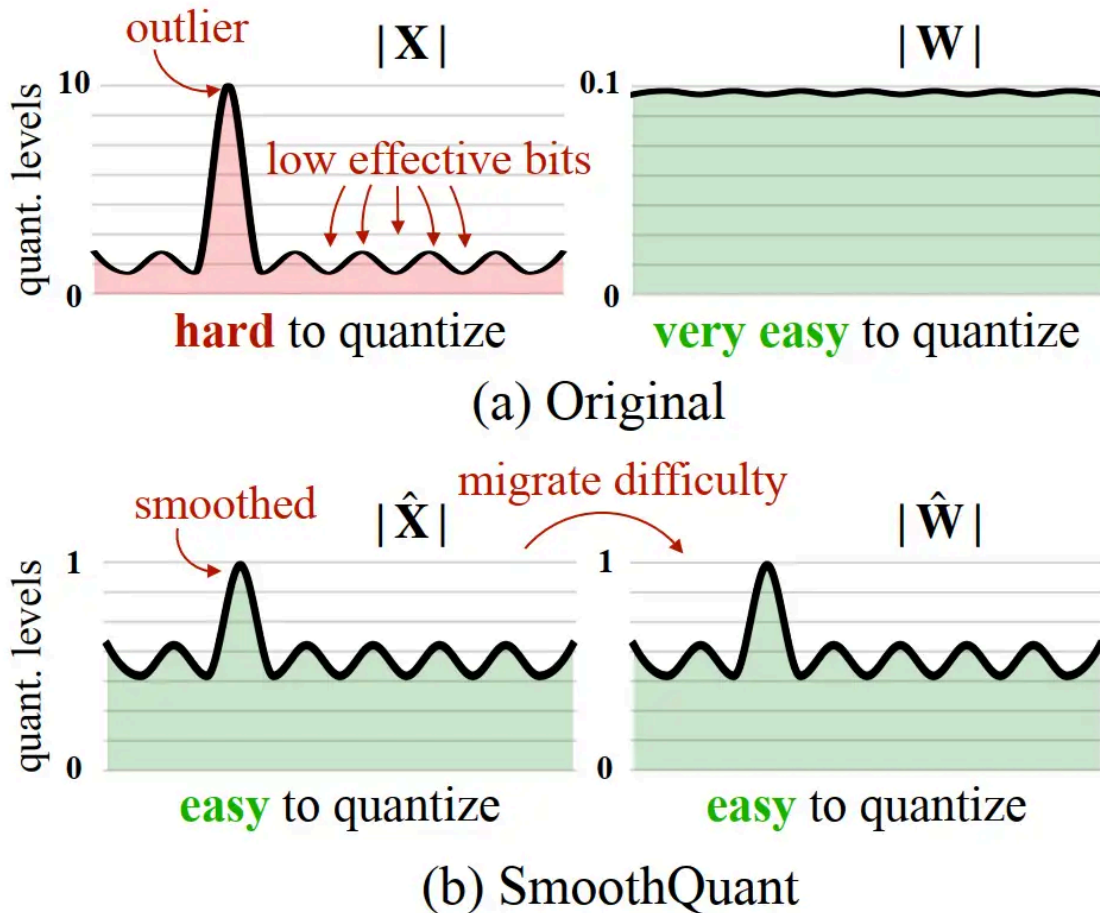


Figure 2: SmoothQuant’s intuition: **the activation X is hard to quantize** because outliers stretch the quantization range, leaving few effective bits for most values. **We migrate the scale variance from activations to weights W during offline to reduce the quantization difficulty of activations.** The smoothed activation $\hat{X}$ and the adjusted weight $\hat{W}$ are both easy to quantize.

@稀土掘金技术社区

需要在 weight 和 activation 中平衡量化难度，让彼此均容易被量化。加入超参 α ： $s_j = \frac{\max(|X_j|)^\alpha}{\max(|W_j|)^{1-\alpha}}$

- 如果 $\alpha = 1$ ：代表把离群点全部抹平，激活值变得极其完美。但所有波动都转移给了权重，导致权重出现严重离群点，权重变得无法量化。
- 如果 $\alpha = 0$ ：什么都不做，退化为传统的 W8A8，激活值无法量化。

- 通常设定 $\alpha = 0.5$ ：折中，激活值的离群点被削弱，权重的波动被微微放大，**两者同时满足了容易使用 INT8 量化的条件。**

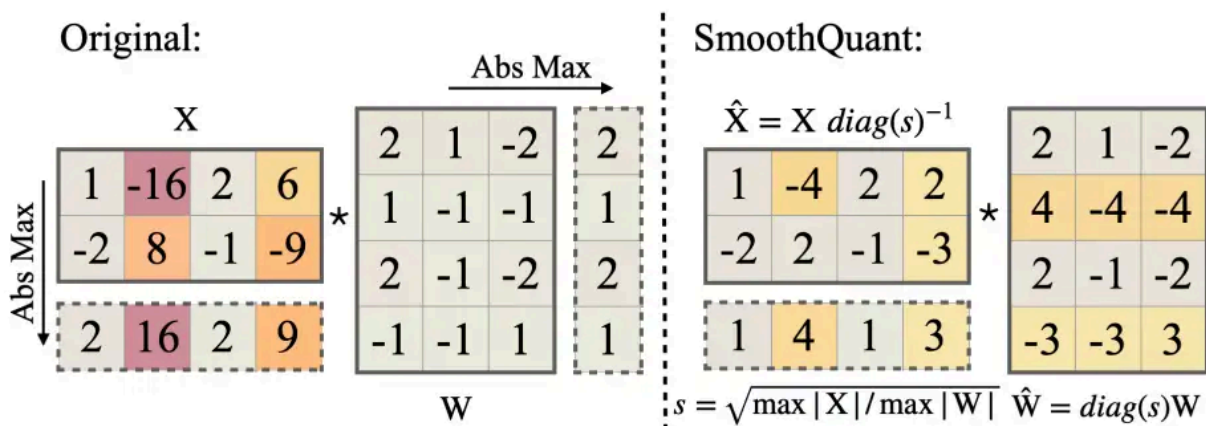


Figure 5: Main idea of SmoothQuant when α is 0.5. The smoothing factor s is obtained **on calibration samples** and the entire transformation is performed offline. At runtime, the activations are smooth **without scaling**.

@稀土掘金技术社区

- Weight-Only 量化 (W4A16, 极低比特)：将权重极端压缩到 4-bit（甚至 3-bit），而计算时仍然反量化回 16-bit 进行激活计算。为突破访存瓶颈和显存容量限制（如 AWQ 和 GPTQ 算法）
- KV Cache 量化：将 KV Cache 量化为 FP8 或 INT4，成倍提升支持的上下文长度和并发 Batch Size

推测解码（Speculative Decoding）

小模型草稿+大模型验证

- 通过Causal Mask，对于长度为K的候选序列，一次前向传播，得到每个位置的下一个token的概率分布（K个概率向量）
- 与草稿模型生成的候选token逐一对比。若拒绝则从该位置开始重新采样。
- 在验证阶段，整个候选序列已经是已知的，可以并行处理，打破自回归串行瓶颈

三、训练与分布式计算

训练原理

Qwen3 前向传播 + 反向传播 + 优化器更新

性能分析

在混合精度训练（Mixed Precision Training, 通常为 FP16/BF16）中，显存开销主要分为四大块：

- **模型权重 (Model Weights)**：模型自身的参数。在混合精度下，通常参数为 16-bit (2 bytes)
- **梯度 (Gradients)**：反向传播计算出的每个参数的导数，同样通常为 16-bit (2 bytes)。
- **优化器状态 (Optimizer States)**：占据显存最大的部分。以 Adam 优化器为例，为保证更新的精度，优化器必须维护一份 32-bit (4 bytes) 的模型权重副本，以及一阶动量 (Momentum, 32-bit, 4 bytes) 和二阶动量 (Variance, 32-bit, 4 bytes)
- **前向激活值 (Activations)**：前向传播计算出的中间结果，必须保存在显存中以便反向传播时计算梯度

显存优化

数据并行 (Data Parallelism, DP)

- 模型规模足够小且单个 GPU 能够承载得下
- 每张 GPU 都必须保存完整的模型参数、梯度和优化器状态
- 把训练数据均分给多个不同的 GPU，让每个 GPU 作为一个计算节点独立的完成前向和反向传播运算

通信原语

- All-Gather：将分布式数据汇总到所有节点，适用于需要全局数据同步的场景
- Reduce-Scatter：将分布式数据进行规约并分散到所有节点，适用于需要局部结果分发的场景
- All-Reduce：Reduce-Scatter 和 All-Gather 的组合。将全局数据规约后同步至所有节点

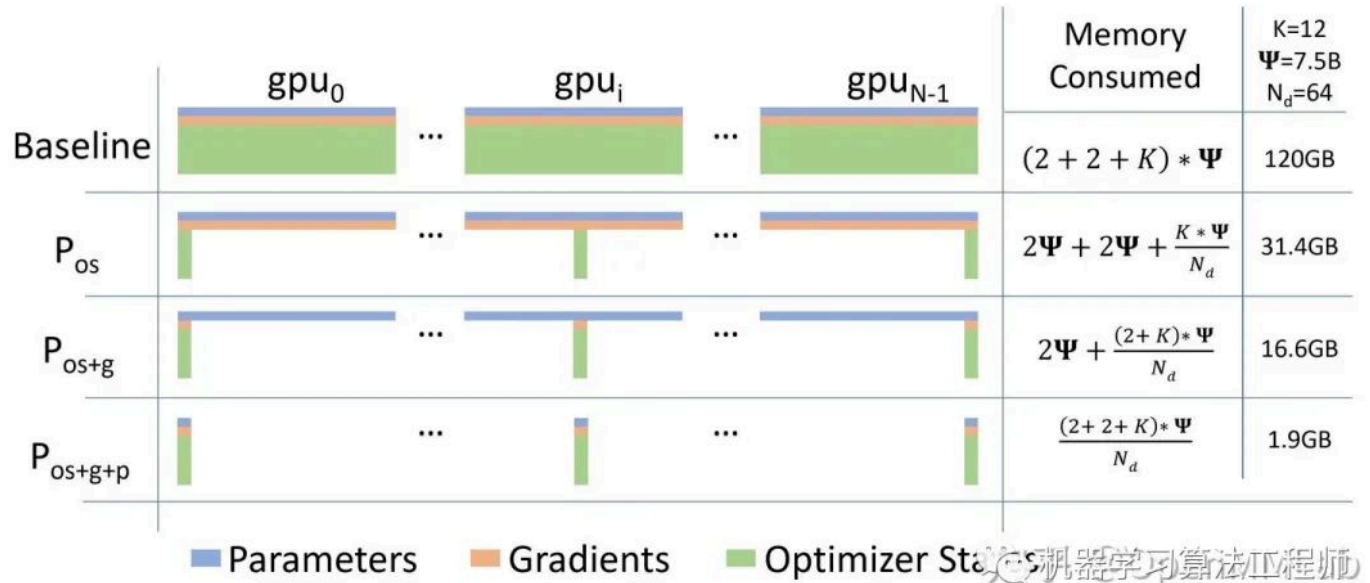
ZeRO (零冗余优化器)

微软 DeepSpeed 团队提出了 ZeRO (Zero Redundancy Optimizer)。

核心思想：消除数据并行中的内存冗余，将参数在各个 GPU 之间进行切分，做到“按需获取”。

- **ZeRO-1 (Optimizer States)**：仅将优化器状态 (OS) 切分到不同的 GPU 上。每张卡只负责更新属于自己的那一块参数。每次迭代结束时，各卡通过集合通信 (All-Gather) 同步更新后的参数。节省约 4 倍的显存，且不增加通信开销。
- **ZeRO-2 (Optimizer States与Gradients)**：在 ZeRO-1 的基础上，进一步将梯度 (G) 进行切分。反向传播时，每张卡只保留自己负责的那部分梯度 (通过 Reduce-Scatter 操作聚合)。节省约 8 倍的显存。
- **ZeRO-3 (全量参数切分)**：不仅切分优化器状态和梯度，还将模型权重 (W) 也切分到各个 GPU。此时单卡上只有极小一部分模型。在进行前向或反向计算时，某一层若需要完整权重，则通

过 All-Gather 临时从其他卡拉取，计算完成后立即丢弃（Free）。这种方式使得显存与 GPU 数量成反比（无限扩展），但代价是极大的通信带宽开销。



激活重算（Activation Recomputation）与 CPU 卸载（Offloading）

- **激活重算 / 梯度检查点（Gradient Checkpointing）：**
 - 激活值通常会占据极大的显存，采用**以时间换空间**
 - 前向传播时，不保存所有的中间激活值，而是只保存少量的“检查点（Checkpoints）”
 - 在反向传播需要用到未保存的激活值时，临时从最近的检查点重新进行一次前向计算来恢复它们
 - 将显存占用从 $O(N)$ 降至 $O(\sqrt{N})$ ，代价是增加约 30% 的计算时间。
- **vLLM - CPU Offloading：**解决 GPU 显存不足的问题
 - Weights CPU Offloading：在模型初始化时将部分模型参数从 GPU 显存转移到 CPU 内存
 - KV Cache CPU Offloading：当 GPU 显存不足以容纳所有请求的 KV Cache 时，vLLM 将暂时不活跃的 KV Cache 块（blocks）从 GPU HBM 卸载到主机内存（CPU DRAM），在需要时再通过 PCIe 带宽换入

大模型分布式并行范式（3D Parallelism）

当模型规模达到千亿（如 GPT-3 175B）甚至万亿规模时，仅靠 ZeRO 已无法提供最佳的计算与通信效率

微软-DeepSpeed 提出 3D 并行（数据并行+张量并行+流水线并行），实现千亿级别模型参数的训练

1. 数据并行（Data Parallelism, DP）

核心思想：将数据分片到多个 GPU，处理不同的数据子集，最后通过通信同步梯度。

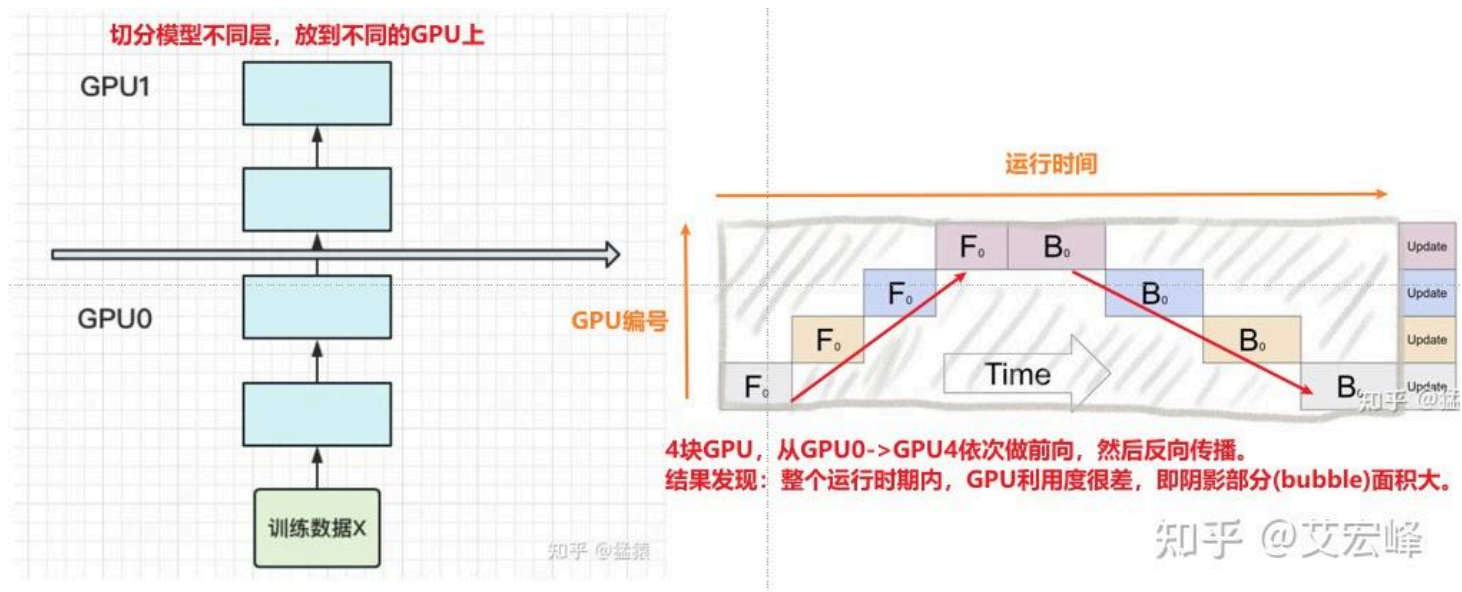
具体实现：

- **传统数据并行（如 PyTorch DDP）**：每个 GPU 独立计算梯度，通过 **All-Reduce** 对所有梯度求和/平均后更新参数。
- **ZeRO（Zero Redundancy Optimizer）**：将优化器状态、梯度和模型参数**分区存储**到不同 GPU，而非每个 GPU 都存一份。结合 Reduce-Scatter（每个 GPU 只负责聚合自己分区的梯度）和 All-Gather（最后收集更新后的参数），通信量与标准 DP 相同（都是 $O(\text{模型参数量})$ ），但显存占用更低。

2. 流水线并行（Pipeline Parallelism, PP）

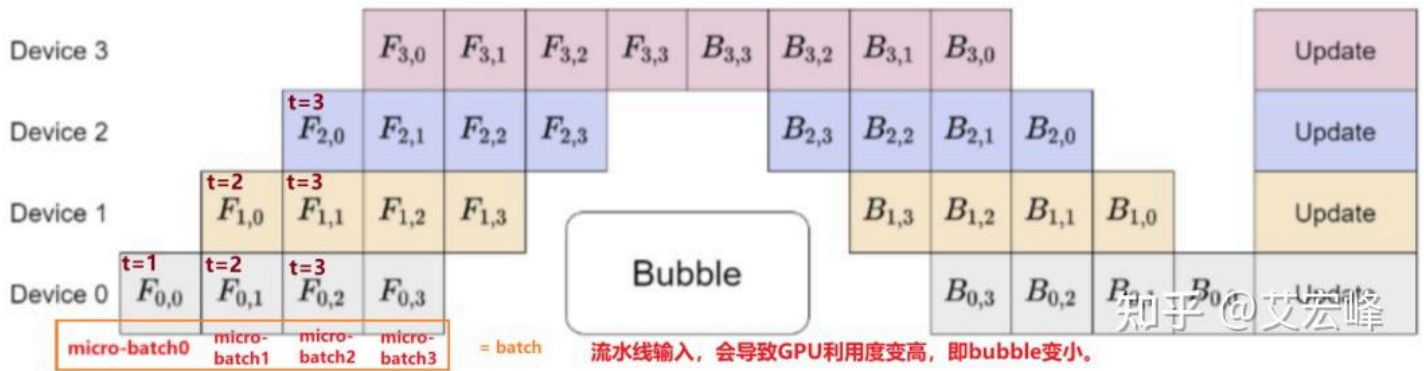
流水线并行是一种**层间并行（Inter-layer Parallelism）**。如 GPU 0 负责 1-5Transformer 层，以此类推

相关问题：在某一时刻只有一张 GPU 在工作，其他 GPU 处于等待上一层输入或下一层梯度的闲置状态，这种闲置时间被称为“气泡”（Bubble）



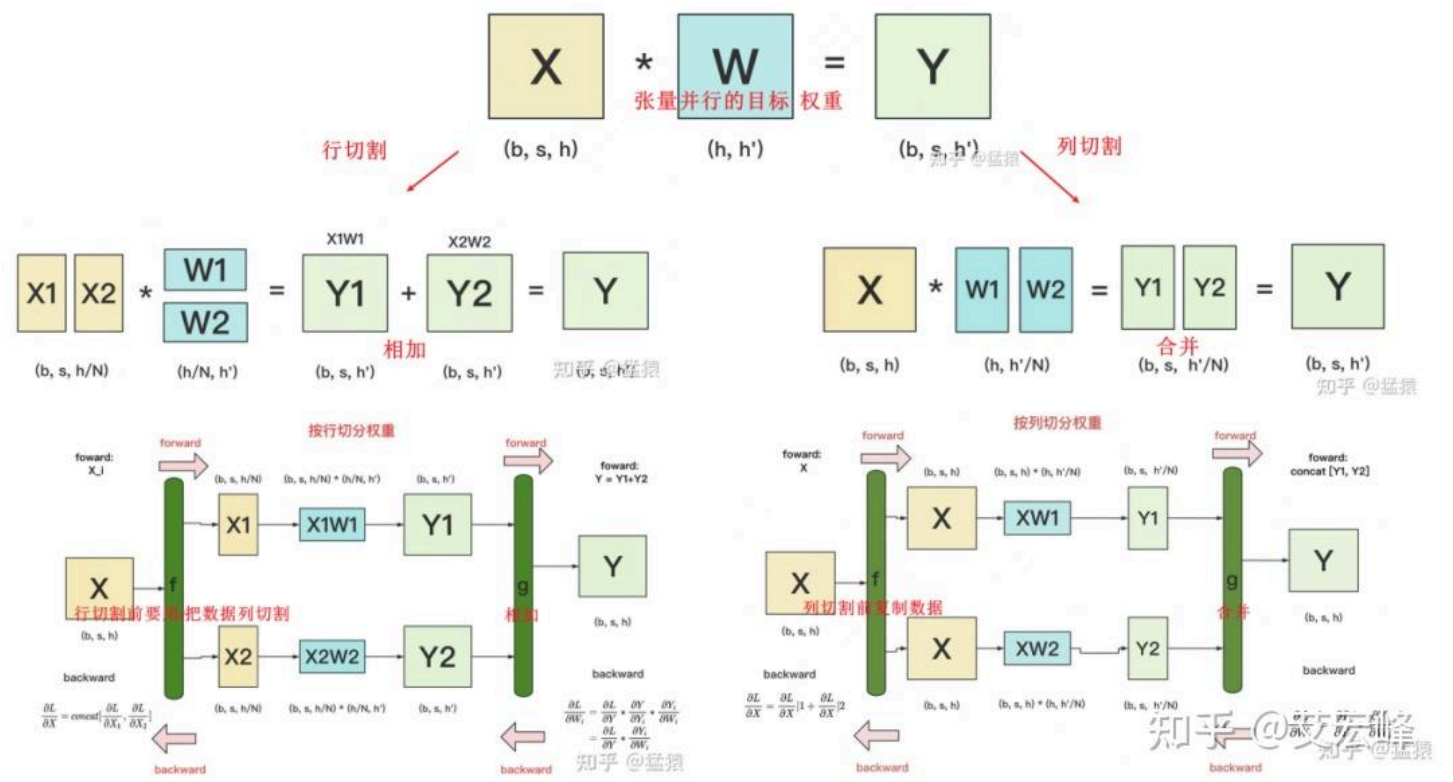
具体实现：

- **GPipe 策略**：将一个大 Batch 切分为多个微批次（Micro-batches），有效填满流水线，减小气泡比例。但 GPipe 需要等所有微批次做完前向，再统一做反向，显存峰值极高
- **Re-materialization (active checkpoint)**：为避免中间结果占用大量显存，只保存来自上一块的最后一层输入 z ，其余中间结果删去



3. 张量并行 (Tensor Parallelism, TP)

张量并行是一种**层内并行 (Intra-layer Parallelism)**，把模型的参数切开，将单个矩阵乘法操作放到不同的GPU上进行独立计算，然后再做聚合



具体实现：

- **列切分 (Column Parallelism)**：将权重矩阵按列切分。多张 GPU 分别拿到完整输入，各自与自己的部分权重相乘，得到部分输出结果。
- **行切分 (Row Parallelism)**：将权重矩阵按行切分。此时输入也需要按列切分，分发给多张 GPU。GPU 各自计算后，需要通过全归约 (All-Reduce) 操作将结果相加。
- **机制与优势**：在 Transformer 的 MLP 块或 Attention 块中，通常将第一层做列切分，第二层做行切分。这样可以使得前向传播在两层之间不需要进行任何跨卡通信，只在模块末尾进行一次 All-Reduce。由于 TP 极度依赖超高带宽的集合通信，通常**只能被限制在单个节点 (Node) 内部**（依赖 NVLink）。

相关资料

领域综述

- AI Infra Introduction – github.io

推理优化

- KV Cache – [SegmentFault](#)
- PagedAttention – [知乎](#)
- Continuous Batching – [知乎](#)
- 量化 – [知乎](#)
- 推测解码 – [知乎](#)

训练并行

- ZeRO – [知乎](#)
- 激活重算 – [知乎](#)
- 3D Parallelism – [知乎](#)
- 3D Parallelism 详解 – github.io